

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **1**
Topic : **2**

Managing Transactions and Data Integrity

Objectives

This session will give the knowledge about

- Understanding essentials of ACID transactions
- Applying transactional guarantee in distributed systems
- Understanding Brewer's CAP Theorem
- Exploring transactional support in NoSQL products

RDBMS and ACID

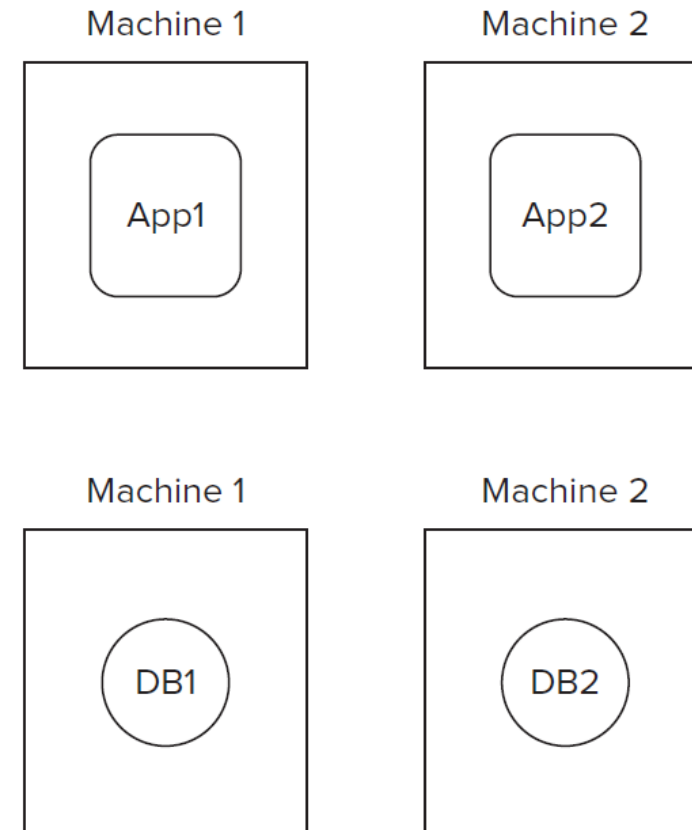
ACID is the key standard and feature of RDBMS

- **Atomicity** – the entire transaction takes place at once or does not happen at all.
- **Consistency** – the database must be consistent before and after the transaction
- **Isolation** – Multiple transactions occur independently without interference
- **Durability** – the changes of a successful transaction occurs even if the system failure occurs.

ACID in Distributed Systems

In distributed systems, the **ACID** principles are applied using the concept of transaction manager or coordinator to manage transactions.

To understand about how ACID properties are achieved in distributed systems, it is important to explore the **properties of distributed systems**



ACID in Distributed Systems

The ACID principles are applied using the concept laid down by the open XA consortium.

It specifies the need for a transaction manager or coordinator to manage transactions.

Even with a central coordinator, implementing isolation across multiple databases is extremely difficult. This is because different databases provide isolation guarantees differently.

Techniques to implement ACID in Distributed Systems

Two-Phase Locking (2PL):

- Ensures isolation by dividing the transaction into a growing phase (acquiring locks) and a shrinking phase (releasing locks).
- Guarantees serializability but may lead to deadlocks.

Strong Strict Two-Phase Locking (SS2PL):

- Extends 2PL by holding all locks until the transaction commits.
- Prevents cascading aborts and ensures a stronger level of consistency.

Techniques to implement ACID in Distributed Systems

Two-Phase Commit (2PC):

- Ensures atomicity in distributed transactions by coordinating commit or abort decisions across all participants.
- Involves a prepare phase (collecting votes) and a commit phase (executing the decision).

Three-Phase Commit (3PC):

- Advance of 2PC to reduce the risk of blocking if the coordinator fails.
- Adds a pre-commit phase to ensure that all participants are ready to commit before making the final decision.

Factors to consider

Assuring ACID-like guarantees in distributed systems is to understand how the following three factors get impacted in such systems:

- Consistency
- Availability
- Partition Tolerance

Consistency, Availability, and Partition Tolerance (CAP) are the three pillars of **Brewer's Theorem** that underlies much of the recent generation of thinking around transactional integrity in large and scalable distributed systems.

Consistency

In the context of CAP it deals with atomicity and isolation.

In ACID, consistency means that data that does not satisfy predefined constraints is not persisted. That's not the same as the consistency in CAP.

Consistency means that all clients see the same data at the same time, no matter which node they connect to. For this to happen, whenever data is written to one node, it must be instantly forwarded or replicated to all the other nodes in the system before the write is deemed 'successful.'

Availability

Availability means the system is available to serve at the time when it's needed.

In terms of CAP

- Even if one or more nodes are down, any client making a data request receives a response. (i.e), when any request is made, without exception, all working nodes in a distributed system return a valid response.

Partition Tolerance

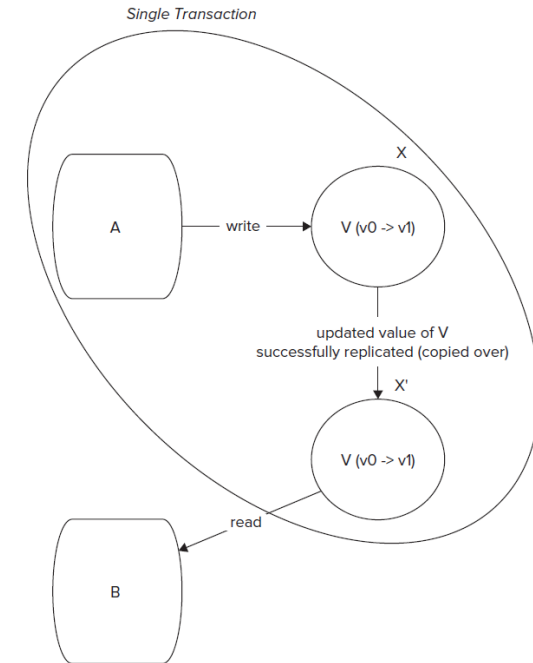
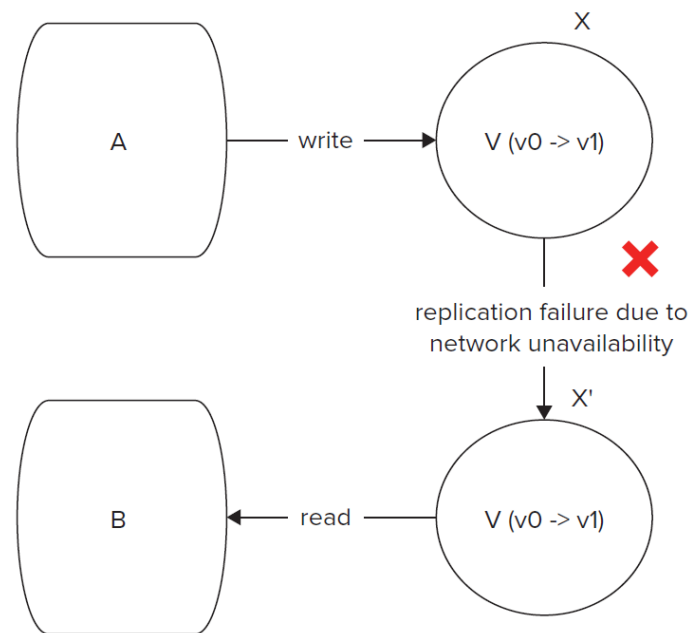
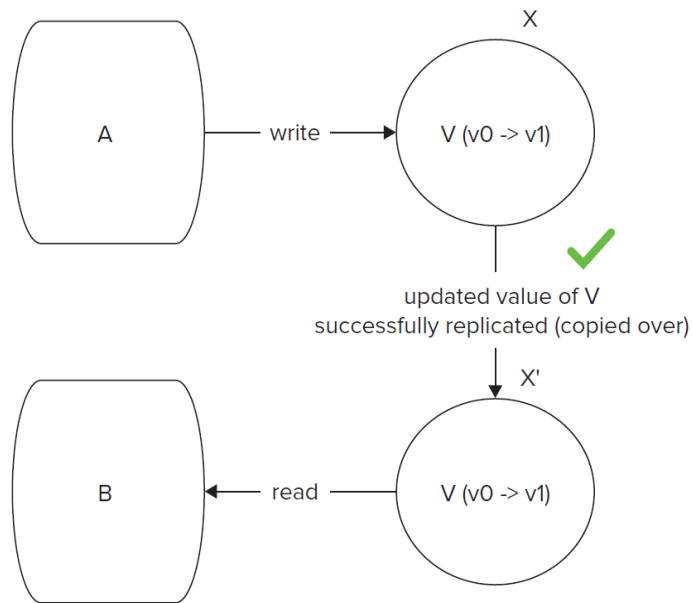
Partition tolerance measures the ability of a system to continue to service in the event a few of its cluster members become unavailable.

A partition is a communications break within a distributed system - a lost or temporarily delayed connection between two nodes. Partition tolerance means that the cluster must continue to work despite any number of communication breakdowns between nodes in the system.

Parallel processing and scaling out are proven methods and are being adopted as the model for scalability and higher performance

Upholding CAP

As proved in Brewer's Theorem, Achieving consistency, availability, and partition tolerance at all times in a large distributed system is **not possible**.



Upholding CAP - Conclusion

The choices could be as follows:

- Option 1 - **Availability is compromised** but consistency and partition tolerance are preferred over it.
- Option 2 - The system has little or **no partition tolerance**. Consistency and availability are preferred.
- Option 3 - **Consistency is compromised** but systems are always available and can work when parts of it are partitioned.

Compromising on Availability

- Compromising on availability means that **the system may not be able to provide a response to every request** in the event of a network partition.
- This trade-off prioritizes consistency and partition tolerance over availability, ensuring that the system remains in a consistent state even if it means some requests cannot be fulfilled immediately.
- **This is common in transactions where failure is not an option, such as money transfers between accounts.** In these cases, systems often have backup and quick recovery mechanisms, like replication, to minimize downtime. Although the system might still be unavailable for a short period, this minor unavailability is usually acceptable.

Compromising on Partition Tolerance

- Compromising on partition tolerance means prioritizing consistency and availability over **the ability to continue functioning during network partitions**. This approach is typically taken when ensuring data accuracy and continuous operation is more important than handling network failures.
- **Google's Bigtable** is a highly available and strongly consistent data store that compromises on partition tolerance. It can handle node failures but not network partitions. In such cases, it identifies primary and secondary parts and tries to resolve issues by forming a quorum.
- **Bigtable and HBase**, along with their underlying filesystems (GFS and HDFS), store three copies of every data set. This ensures consistency by consensus, allowing the system to remain consistent even if one copy fails or falls out of sync. Fewer than three copies would not guarantee consensus.

Compromising on Consistency

- Compromising on consistency involves prioritizing availability and partition tolerance over ensuring that all nodes always have the same data. This approach is often used in systems where it is more important to remain operational and responsive, even if some nodes temporarily have outdated or inconsistent data.
- Example-1: A shopping cart could **allow orders even if it's not able to confirm** with the inventory system about availability. In a possible limit case, the product ordered may be out of stock. In such a case, the order could be taken as a back order and filled when the inventory is restocked.
- Example-2: A bank could **allow a customer to withdraw up to an overdraft amount** even if it's unable to confirm the available balance so that in the worst situation if the money isn't sufficient the transaction could still be valid and the overdraft facility could be used.

Eventual Consistency

NoSQL databases (for example, ScyllaDB) use a model of **tunable eventual consistency** to deliver multi datacenter high availability and fast and efficient write and read operations.

In this case, all nodes are equal; this means any node can serve any request, there is no single point of coordination, and all nodes in the system continue to cooperatively provide service, even when nodes become unavailable.

Eventual consistency supports modern workloads that are **less dependent on strong consistency but rely heavily on availability**.

Compromising on Consistency

To understand **eventual consistency** one may try to illustrate a situation in terms of the following three criteria:

- R — Number of nodes that are read from.
- W — Number of nodes that are written to.
- N — Total number of nodes in the cluster.

Different combinations of these three parameters can have different impacts on overall consistency. **Keeping $R < N$ and $W < N$ allows for higher availability.**

Compromising on Consistency

Following are some common situations

- $R + W > N$ — In such situations, a consistent state can easily be established, because there is an overlap between some read and write nodes.
- $R = 1, W = N$ — When a system has more reads than writes, it makes sense to balance the read load out to all nodes of the cluster.
- $R = N, W = 1$ — When writing to one node is enough, the chance of data inconsistency can be quite high.
- $R = W = \text{ceiling}((N + 1)/2)$ — Such a situation provides an effective quorum to provide eventual consistency.

Conclusion

Strong consistency suggest declined to consider NoSQL databases seriously due to its relaxed consistency configurations.

Though consistency is an important requirement in many transactional systems, the strong-or-nothing approach has created a lot of fear, uncertainty, and doubt among users.

Eventual consistency has its place and should be used where it safely provides high availability under partition conditions.

Summary

This session will give the knowledge about

- Understanding essentials of ACID transactions
- Applying transactional guarantee in distributed systems
- Understanding Brewer's CAP Theorem
- Exploring transactional support in NoSQL products